CS & IT ENGINEERING



Algorithms

Divide & Conquer



Lecture No.- 05

By- Aditya sir

Recap of Previous Lecture



- Topic
- Topic

Merge Sort

↳ TC, SC, Code, Ex

Topics to be Covered



- Topic
- Topic
- Topic

Quick Sort



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.

Telegram

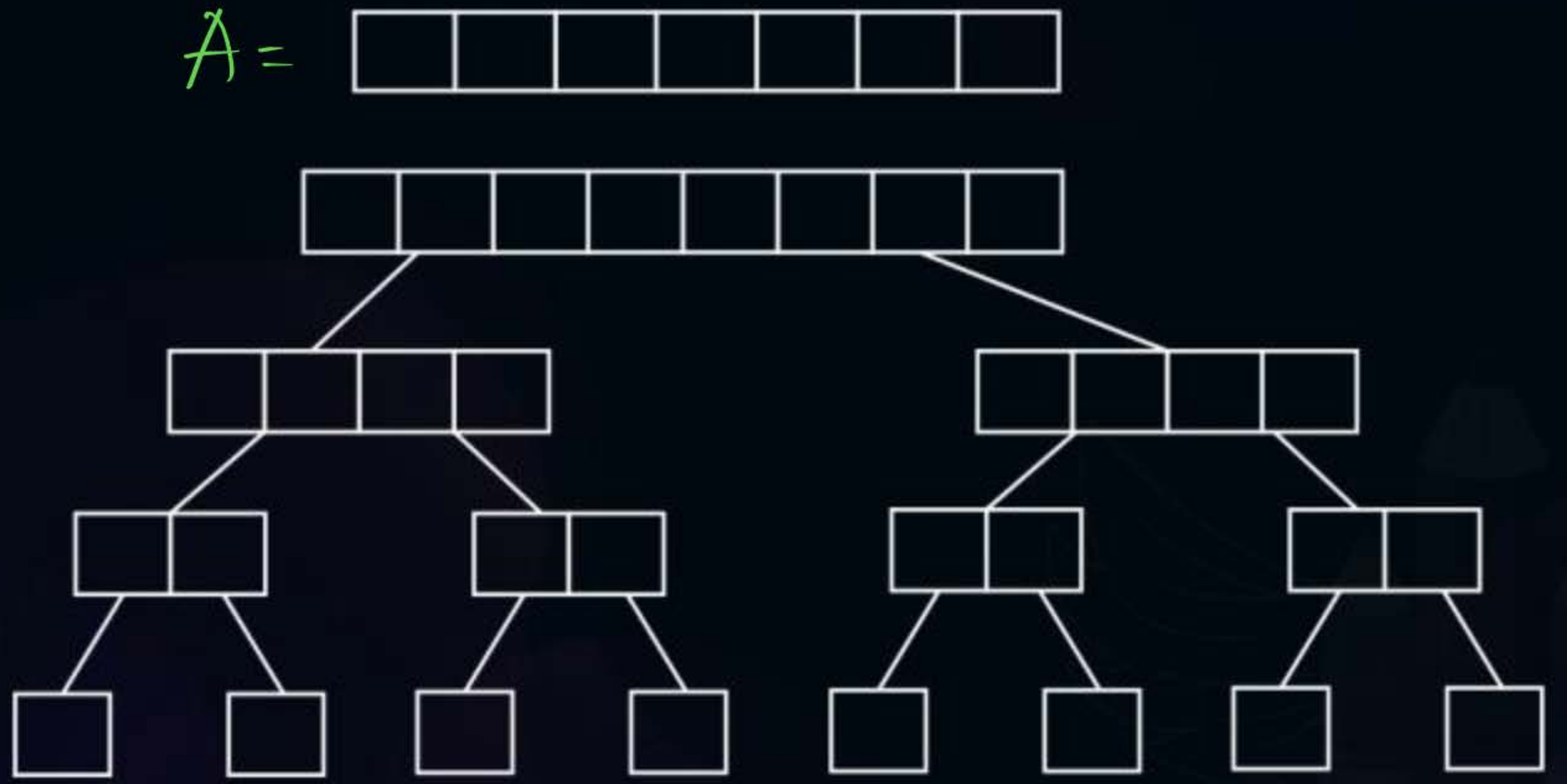


Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW

Merge Sort : 2-way MS (Bottom-up)



Topic : 2 – Way Merge Sort (Bottom up merge Sort)



#Q. If one uses straight two-way merge sort algorithm to sort the following elements in ascending order:

20, 47, 15, 8, 9, 4, 40, 30, 12, 17

then the order of these elements after second pass of the algorithm is:

A. 8, 9, 15, 20, 47, 4, 12, 17, 30, 40

B. 8, 15, 20, 47, 4, 9, 30, 40, 12, 17

C. 15, 20, 47, 4, 8, 9, 12, 30, 40, 17

D. 4, 8, 9, 15, 20, 47, 12, 17, 30, 40



Topic : Divide and Conquer

[20, 47, 15, 8, 9, 4, 40, 30, 12, 17]

pass 4 o/p: Sorted → 4 8 9 12 15 17 20 30 40 47

pass 3 o/p : 4 8 9 15 20 30 40 47 | 12 17

pass 2 : 8 15 20 47 | 4 9 30 40 | 12 17

pass 1 : → 20 47 | 8 15 | 4 9 | 30 40 | 12 17

Start → 20 | 47 | 15 | 8 | 9 | 4 | 40 | 30 | 12 | 17



Topic : Quick Sort

Quick Sort is based on partitioning algorithm/Process.

70%

D & C

D & C



Topic : Partitioning Process:- (Divide)

V. Imp

Given:- A: 65, 70, 75, 80, 85, 60, 55, 50, 45

1 2 3 4 5 6 7 8 9

Pivot

(60 55 50 45) 65 (70 75 80, 85)

1 ⟶ 4 5 6 ⟶ 9

Steps:- usually, we take the first element as pivot. (default)

(1) find the correct place of Pivot element in the final sorted List.

(2) After the pivot is placed at its correct position, all the elements that are lesser than pivot, will be on its left. (after Partitioning)

(3) And, similarly, all the elements that are greater than pivot, will be on its right.



Topic : Divide and Conquer

```
1.  Algorithm Partition(a,m,p)
2.  // Within a[m],a[m + 1],....., a[p – 1] the elements are
3.  // rearranged in such a manner that if initially t = a[m],
4.  // then after completion a[q] = t for some q between m
5.  // and p – 1, a[k] ≤ t for m≤ k < q, and a[k] > t
6.  // for q< k < p. q Is returned. Set a[p] = ∞.
7.  {
8.      v : = a[m]; i := m; j := p;
9.      repeat
10.     {
```

O(n)

a[m ⟶ p-1]

p
∞

pivot = t

m ≤ q ≤ p-1

Correct pos of pivot

(<) q (>)

m q-1 q+1 p-1



Topic : Divide and Conquer

```
11.        repeat
12.        i: = i + 1;
13.        until (a[i] ≥ v);
14.        repeat
15.        j: = j - 1;
16.        until (a [j] ≤ v);
17.        if (i < j) then Interchanged (a,i, j);
18.      } until (i ≥ j);
19.      a[m] := a[j]; a[J] := v; return j;
20.    }
```



Topic : Divide and Conquer

```
1.  Algorithm Interchanged(a, i, J)
2.  // Exchange a[i] with a[j].
3.  {
4.        p := a[i];
5.        a[i] := a[j]; a[j] := p;
6.  }
```

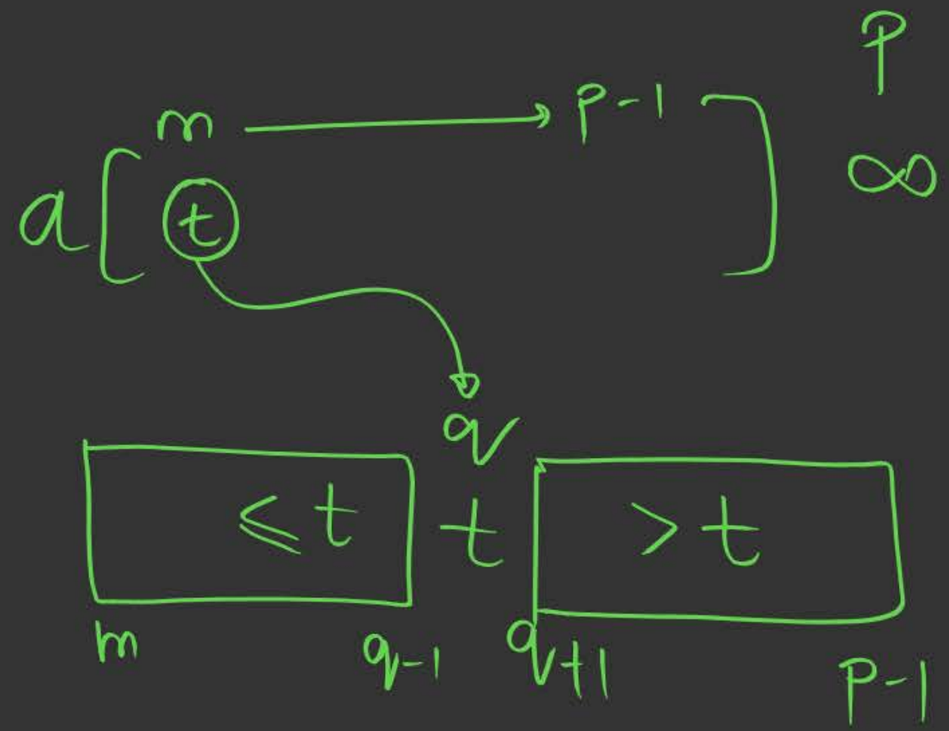
Swap(a(i), a(j))

Explanation of code:

$t = a[m]$

↓

Pivot



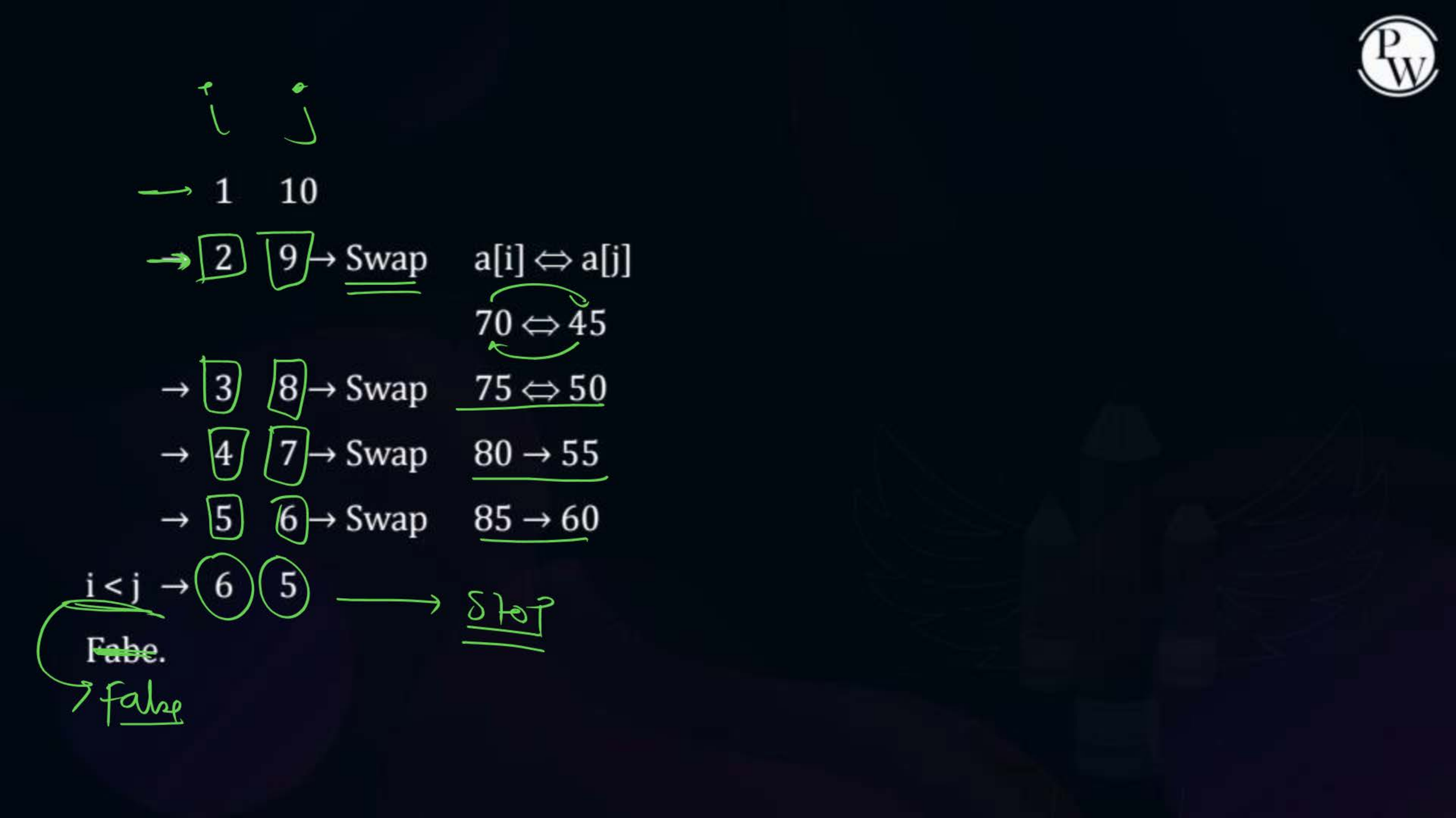
QS

↳ Divide (Partition)

Working of Partition Algo

Given:-




i j
1 10
2 9 → Swap a[i] ⇔ a[j]
70 ⇔ 45
3 8 → Swap 75 ⇔ 50
4 7 → Swap 80 → 55
5 6 → Swap 85 → 60
i<j → 6 5
Stop
False.
False



Topic : Divide and Conquer

a[p ——→ q]

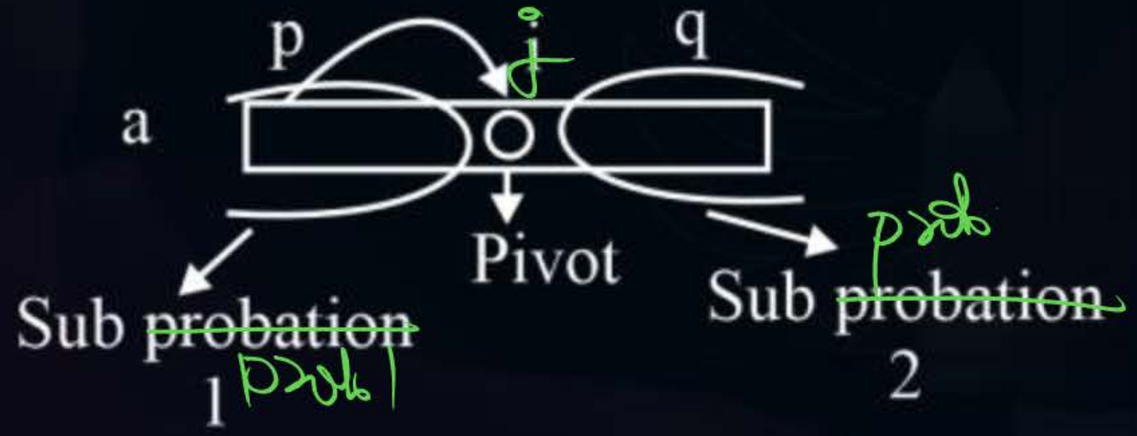
```
1.  Algorithm QuickSort(p, q)
2.  // Sorts the elements a[p],..., a[q] which reside in the global
3.  // array a [1: n] into ascending order: a[n + 1] is considered to
4.  // be defined and must be > all the elements in [l: n].
5.  {
6.       if (p < q) then // If there are more than one element
7.       {
8.            // divide P into two subproblems.
9.            j := Partition(a, p, q + 1);
```

TC = O(n)

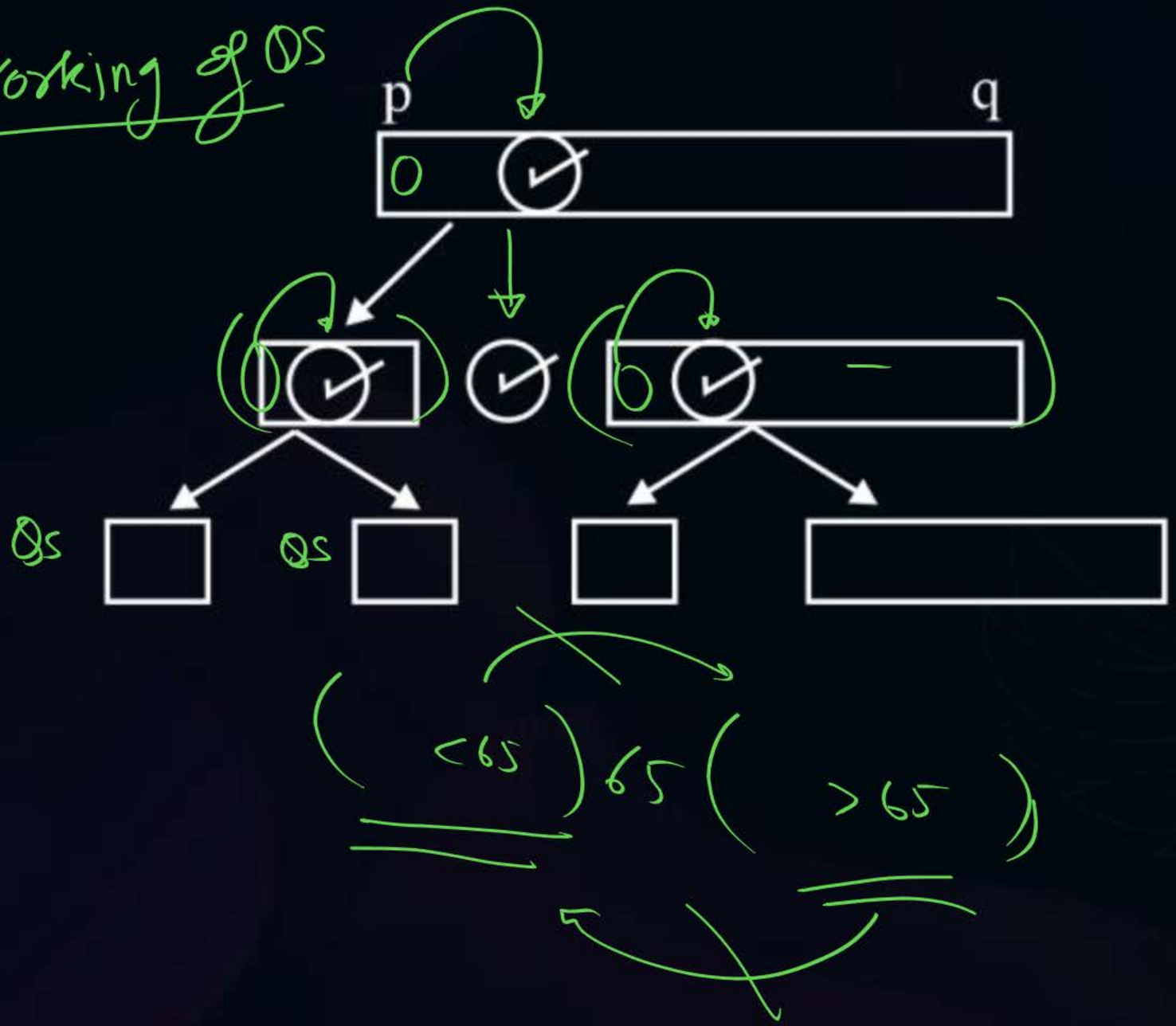


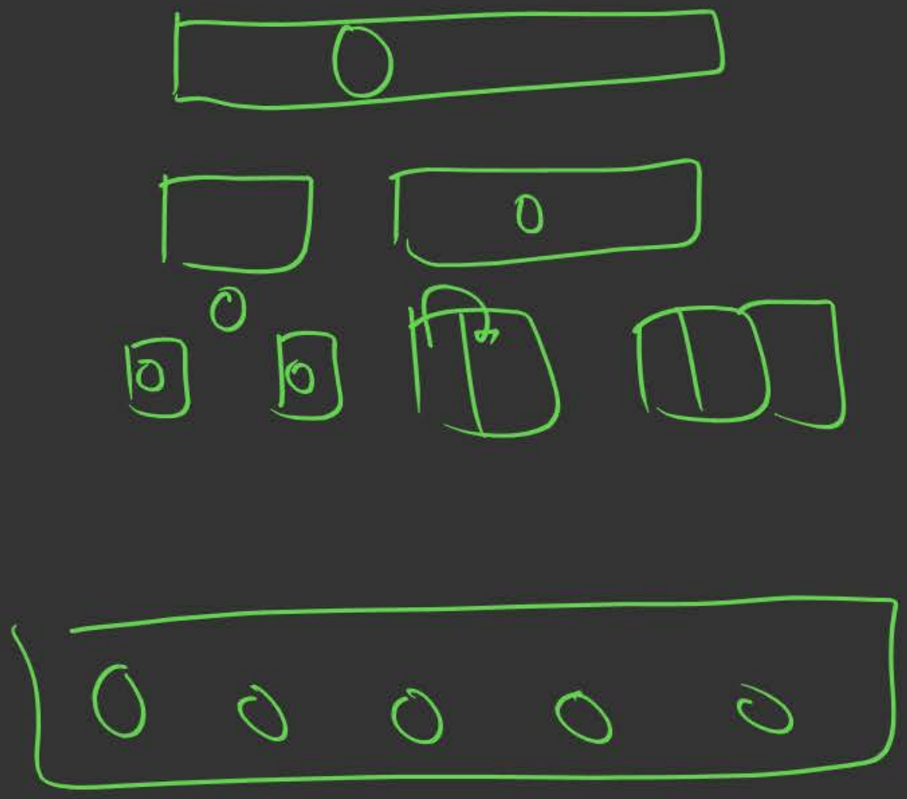
Topic : Divide and Conquer

```
11.        // Solve the subproblems.
12.        QuickSort(p, j – 1);
13.        QuickSort(j + 1,q);
14.        //There is no need for combining solutions.
15.    }
16. }
```



Working of QS







Topic : Divide and Conquer

Performance:- Complexity analysis of ~~Quiche~~ Quick Sort

Problem :

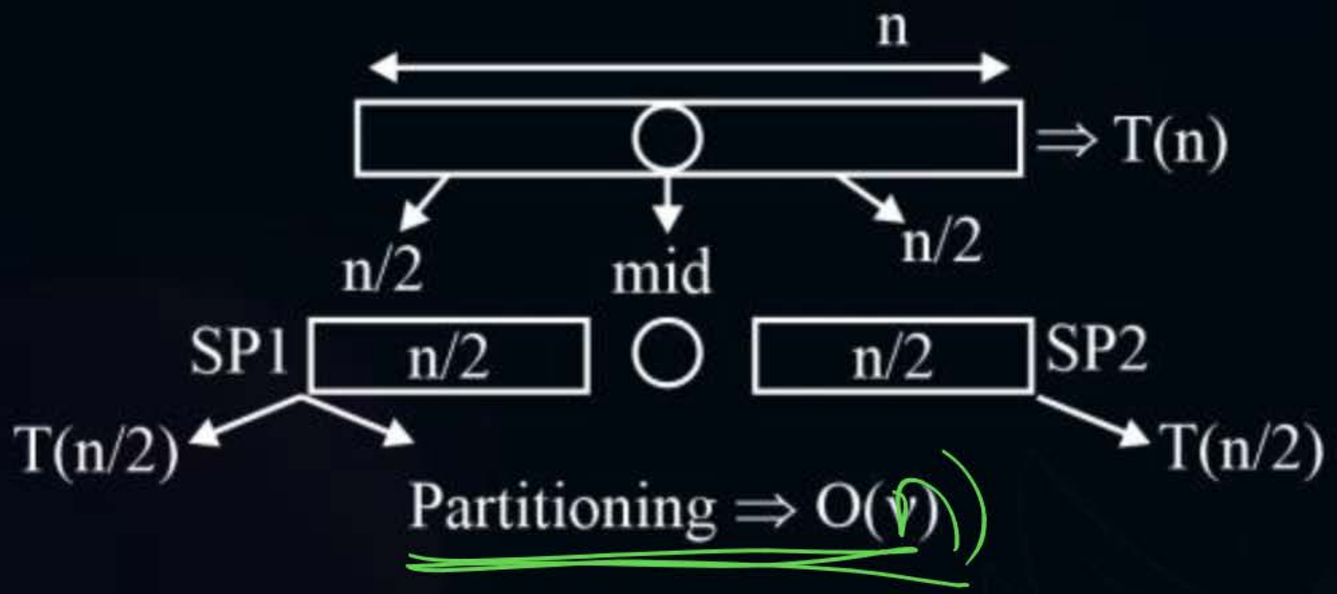


Case 1: After partitioning, pivot always gets placed at the middle position, thereby dividing the array into 2 nearly almost equal pasts.

↳ TC ?



Topic : Divide and Conquer



Case 1: Time Complexity:-

$$T(n) = T(n/2) + T(n/2) + O(n)$$

partitiong

left Sub-problem

right Subproblem



Topic : Divide and Conquer

$T(n) = 2T(n/2) + O(n)$

$T(n) = 2T(n/2) + n$

$T(n) = O(n\log_2 n)$

Same as Merge Sort Recurrence



Topic : Divide and Conquer

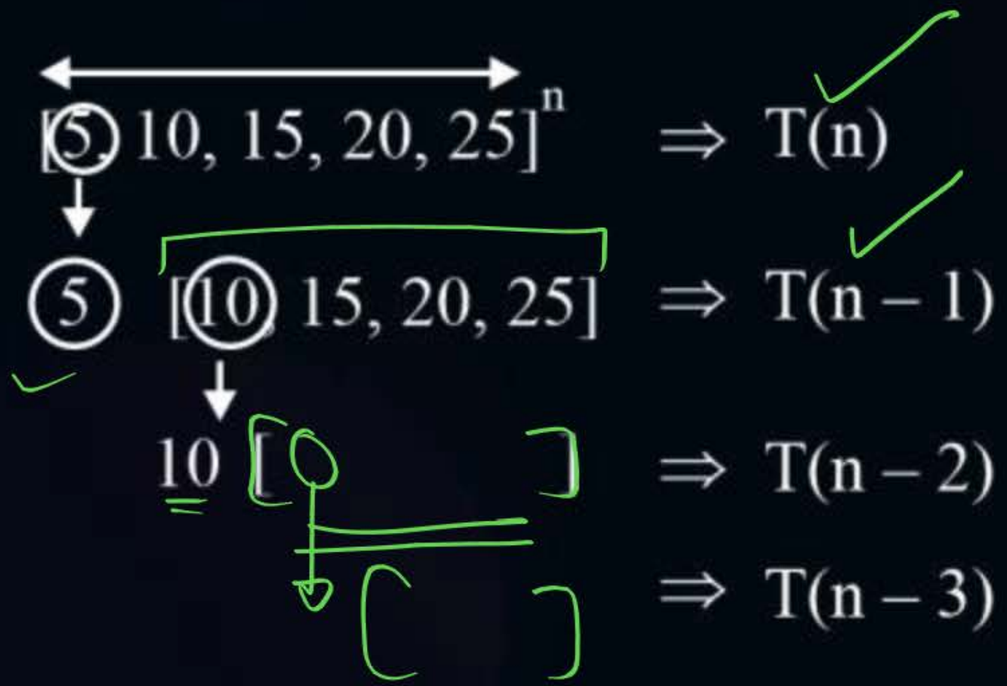
Case 2:- When after partitioning, the pivot gets placed always at the 1st position or the last position, there by reducing the size of the problem by just 1.

When array is already sorted in decrease or increase order.

Default → 1st elem is pivot

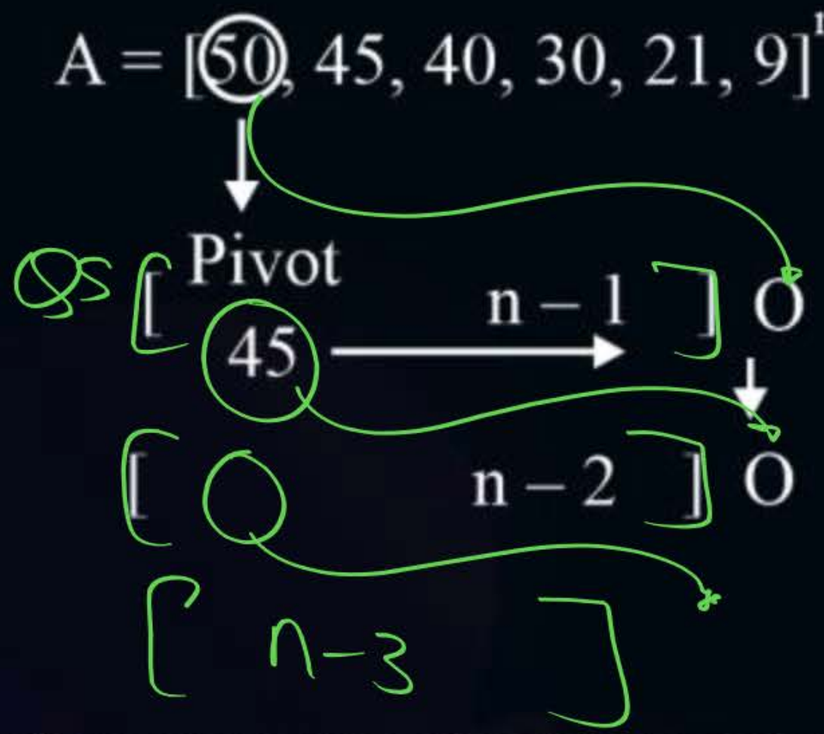
Input :-

eg :- asc



Note :- This happens when the array is already ~~looted~~ sorted in ascending order.

Input 2 :- $A = [50, 45, 40, 30, 21, 9]^n$



$T(n) = T(n-1) + O(n)$

Note :- This happens when input array is already sorted in descending order.

Time Complexity Recorder for Case 2

(Input or Input 2)

$T(n) = T(n-1) + O(n)$ → partitioning

Subproblem

$T(n) = T(n-1) + n$

$= T(n-2) + n + (n-1)$

$= T(n-3) + n + (n-1) + (n-2)$

$= T(n-k) + n + (n-1) \ldots\ldots + (n-(k-1))$

Gural form

$T(n) = T(n - k) + n + (n - 1) \ldots\ldots (n - (k - 1))$

For base condition,

$N - k = 1 \Rightarrow k = (n - 1)$

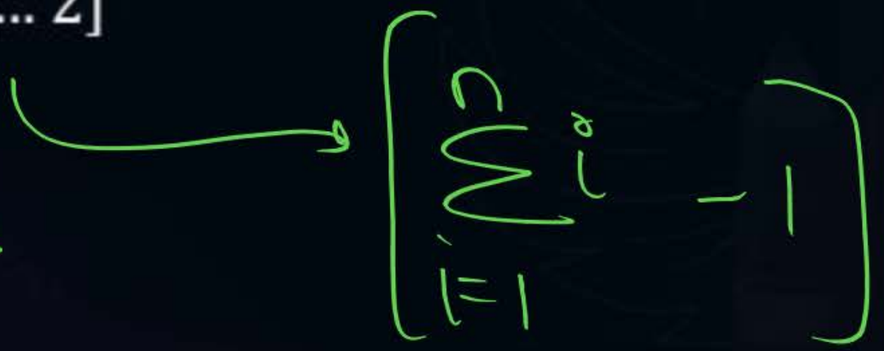
$T(n) = T(1) + n + (n - 1) \ldots\ldots (n - k + 1)$

$= T(1) + [n + (n - 1) + (n - 2) \ldots\ldots 2]$

↓ ↓

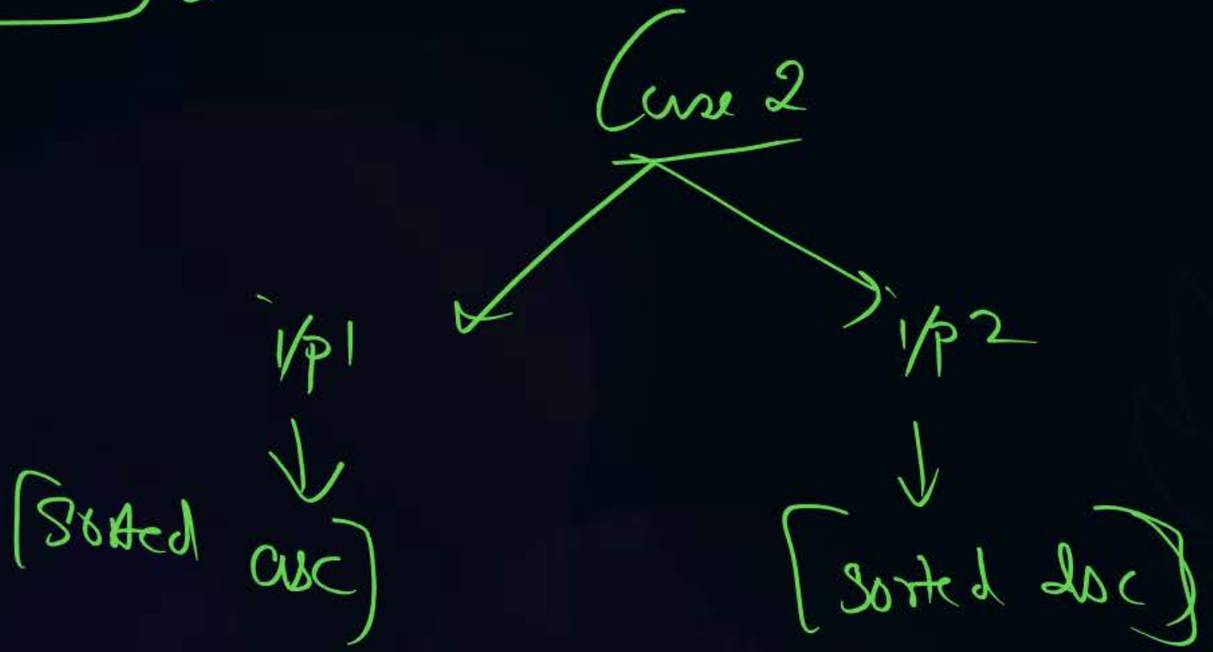
Const





$$T(n) = C + \frac{n(n+1)}{2} - 1$$

$$T(n) = O(n^2)$$

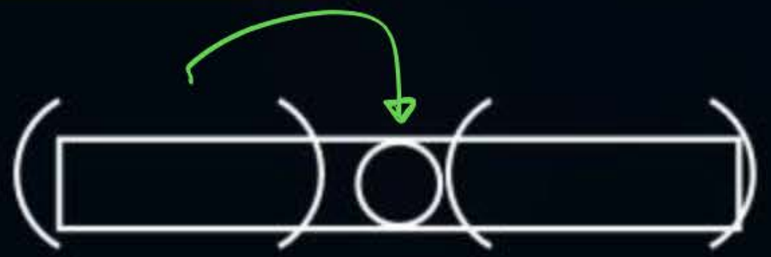




Topic : Time Complexity Sammary

(1) Bast Case:- Input is unsorted

$TC = \Omega(n \log n)$ $TC = O(n \log_2 n)$



(2) Worst Case:-



array is in descending order

when input is already sorted ~~case or disc~~ in asc

$TC = O(n^2)$

In general

$TC = O(n \log n)$



Topic : Space Complexity

SC

Best Case :- $O(\log_2 n)$

~~n/2k~~

$\frac{n}{2^k} = 1$

$k = \log_2 n$



Recursion stack

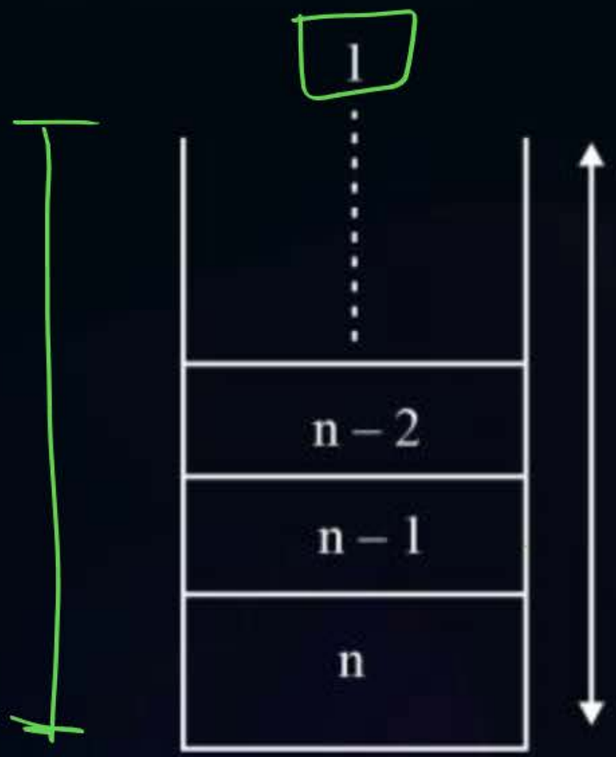
$O(\log_2 n)$



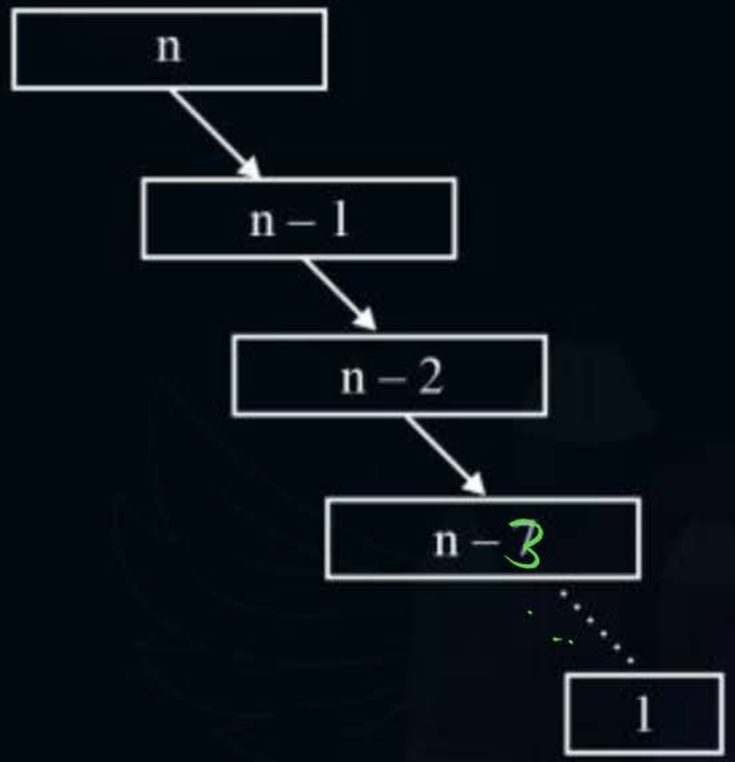


Topic : Worst Case

Recursion Stack



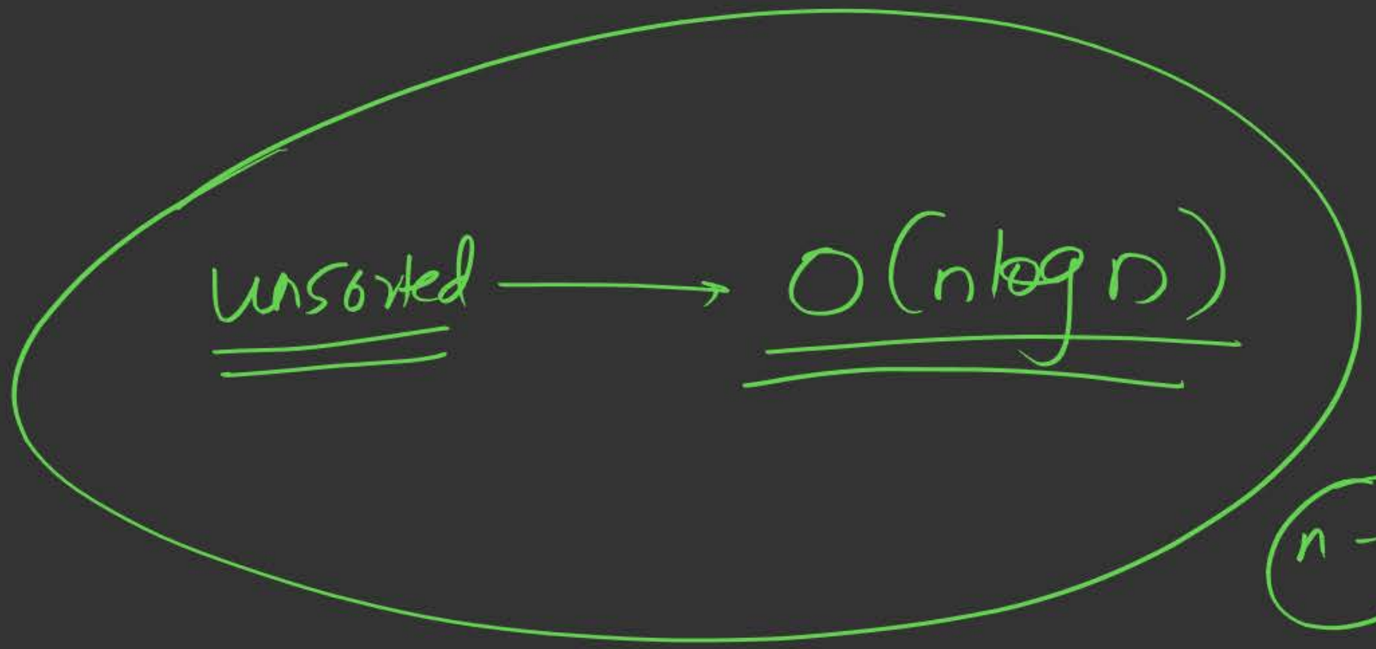
⇒ O(n)
worst case
Space Complaxity



1) Is Quick Sort inplace ?

↳ Yes * Imp

2) Is QS stable ⟶ No

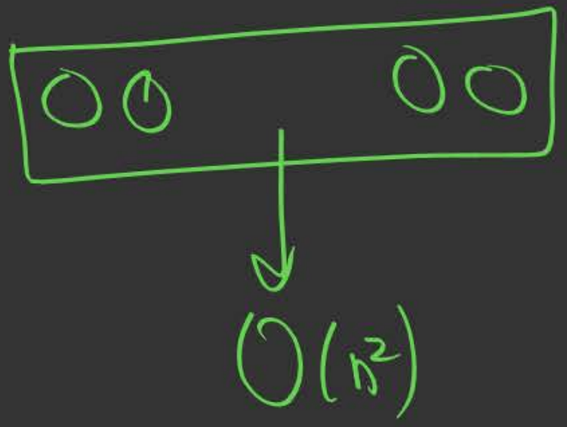


n - 1

QS :- BC → O(n log n)

AC → O(n log n)

WC → O(n²) → asc / dsc



H.W

↳ Revise this lec Content.



2 mins Summary

- Topic
- Topic
- Topic
- Topic
- Topic

Quick Sort



THANK - YOU